

EdgeAI - Specialization & Visualization

Mutlu, Berat Emir - Kocyigit, Kazim Efe

Contents

1	Introduction	2
1.1	Project Information	2
1.2	Goals and Content	2
1.3	Project Organization	2
1.4	Structure of the Report	2
2	Team Tasks and Responsibilities	2
2.1	Project Planning	2
2.2	Implementation	2
2.3	Results	2
2.4	Summary	3
2.5	Next Steps	3
2.6	References	3
3	Specialized Tasks	3
3.1	Color Detection	3
3.2	Direction Detection	4
4	Web-Based Visualization Tool	4
4.1	Purpose and Functionality	4
4.2	User Interaction	4
4.3	Chart Design	5
4.4	Code Structure	5
5	Conclusion	5

1 Introduction

1.1 Project Information

We (Group 2) were responsible for two main parts of the project:

- Developing specialized tasks (color and direction detection) as part of the federated learning framework.
- Developing a web-based graphical user interface (GUI) to visualize the outputs of these tasks in an interactive manner.

1.2 Goals and Content

The goal of this project is to implement a real-time image processing tasks under a federated learning environment. The specialized tasks realize detection and analysis of object attributes (e.g., color and direction), while the web interface provides visual insight into our model behavior.

1.3 Project Organization

Our supervisor distributed the responsibilities across subgroups, with Group 2 focusing on the specialization and visualization components. We had regular meetings every week with Group 2 to ensure synchronization across development milestones.

1.4 Structure of the Report

This report has 4 following sections.

- Team tasks and implementation details
- Specialized edge-device functionalities
- Visualization tool
- Summary

2 Team Tasks and Responsibilities

2.1 Project Planning

Initial planning involved defining concrete tasks, allocating team members, and setting up shared repositories for collaboration.

2.2 Implementation

Our main tasks were:

- Color detection using image processing and clustering.
- Direction detection using geometric vector estimation.
- Web-based interactive visualization.

2.3 Results

We implemented specialized tasks and the frontend tool successfully. Color and direction detection make all-day usage of the model broader and more suitable. Data visualization helps us to validate and debug the model efficiently.

2.4 Summary

We have delivered functionalities that are suitable for the larger federated learning system.

2.5 Next Steps

We could add more kinds of visual features and make the frontend allow more advanced ways for users to interact.

2.6 References

We listed the references used throughout development in Section 5.

3 Specialized Tasks

3.1 Color Detection

Objective: Detecting the dominant color of each tracked vehicle.

Methodology:

1. Detecting vehicles using YOLOv8.
2. Extracting of bounding box regions with `cropVehicle()`.
3. Getting rid of transparent pixels using RGBA masking.
4. RGB pixel clustering using K-Means.
5. Matching the dominant RGB value to an XKCD color label (e.g., *light blue*, *dark red*).
6. Updating the detected color label periodically.

Python Implementation:

Listing 1: Color Detection via KMeans Clustering

```
def cropCar(image):
    .
    .
    .
    largest = max(contours, key=cv2.contourArea)

    mask = np.zeros(image.shape[:2], dtype=np.uint8)
    cv2.drawContours(mask, [largest], -1, color=(255,), thickness=-1)

    masked = cv2.bitwise_and(image, image, mask=mask)
    x, y, w, h = cv2.boundingRect(largest)
    car_cropped = masked[y:y + h, x:x + w]
    mask_cropped = mask[y:y + h, x:x + w]

    if car_cropped.shape[0] == 0 or car_cropped.shape[1] == 0:
        print("Cropped image is empty.")
        return None, None, None

    b, g, r = cv2.split(car_cropped)
    alpha = mask_cropped
    rgba = cv2.merge((r, g, b, alpha))
```

3.2 Direction Detection

Objective: Detecting the movement direction of each tracked vehicle.

Methodology:

1. Computing the center point of each tracked vehicle per frame.
2. Determining the motion angle based on the last two frames.
3. Matching angle to one of the eight directions (N, NE, E, etc.).
4. Updating direction label periodically.

Python Implementation:

Listing 2: Direction Estimation Using Angle Between Frames

```
def get_center(x1, y1, x2, y2):  
    return int((x1 + x2) / 2), int((y1 + y2) / 2)  
  
def compute_angle(start, end):  
    dx = end[0] - start[0]  
    dy = end[1] - start[1]  
    return math.degrees(math.atan2(-dy, dx)) % 360
```

4 Web-Based Visualization Tool

4.1 Purpose and Functionality

The tool uses a JSON file to display ML predictions based on the footage:

- Frame metadata
- Object class predictions
- System usage stats (CPU, memory)

Main Features:

- Tracking classification changes over time
- Observing model behavior frame-by-frame
- Comparing object class frequencies
- Viewing hardware usage charts

4.2 User Interaction

- Dropdowns to pick single or multiple charts at a time
- Buttons to select which part of the project to be displayed(e.g. Specialization, Model Developing)
- Hover to view exact values

4.3 Chart Design

Technology: `Chart.js`

Chart Types:

- Line Chart: prediction confidence over time
- Bar Chart: object class frequency
- Time-series charts: CPU/RAM usage

We styled charts within HTML card containers for a clean and organized layout.

4.4 Code Structure

- `index.html`: layout, canvas, and external links
- `script.js`: data parsing, chart setup, interactivity
- `style.css`: layout styling and UX behavior
- `assets/`: contains images used

5 Conclusion

We have implemented and tested real-time visual perception modules for edge devices and provided an interactive frontend for monitoring their output. These tools contribute to interpretable federated learning.

References

- YOLO: <https://github.com/ultralytics/ultralytics>
- Chart.js: <https://www.chartjs.org/>
- XKCD Color List: <https://xkcd.com/color/rgb/>